Data Creators › instructor pack — LLM guide

The LLM guide: Claude on this laptop, Copilot in the room

Learners use Copilot Chat in VS Code with the AI Catalyst plugin, inside the training dev container. The instructor uses Claude Code on this laptop, with the same Catalyst skills installed as Claude skills. The prompts are the same; only the way a skill is invoked differs. Say that once on Day 1 and then show both side by side whenever it matters.

Same skills, two harnesses

	Learners (Copilot)	**Instructor (Claude Code)**
Where	VS Code Copilot Chat, in the dev container	A terminal in the `courseware/` folder, or Claude Code's VS Code panel with `courseware/` open
Invoke a skill	`/ai-catalyst product-requirements-document <description or pasted transcript>` or `/product-requirements-document …`	`/product-requirements-document <description or pasted transcript>` (installed by `courseware/common/catalyst/install-skills.sh`)
See the skill's instructions	The plugin's skill file; in the Catalyst repo under `skills/<name>/SKILL.md`	`courseware/.claude/skills/<name>/SKILL.md` and its `templates/`
Model	The model picker (Claude and GPT families; the names change month to month)	`/model`
Reasoning effort	"Thinking effort" slider	`/model` also sets the effort level
Modes	Ask, Edit, Agent; approvals default, autopilot	Claude Code is agentic by default; it asks before edits unless auto-approved
Context	Grows with the chat; compaction happens automatically near the limit	`/compact` and `/clear`; same advice: summarise to a file and start fresh

© 2026 Decoded Limited · decoded.com

Four lines to say on Day 1 when the interface comes up: the model is the engine, the harness is the car; a skill is a saved prompt the harness loads; a tool call is JSON the harness executes; what you see as “thinking” is a summary of hidden tokens you pay for.

What to generate live and what to have ready

Beat	Live with Claude	Ready in `courseware/common/running-example/`
Day 1 PRD from a transcript	Yes: paste `interview-transcript.md` into `/product-requirements-document`	`prd.md`
Day 1 data contract	Yes: `/data-contract` with the PRD in context	`data-contract.md`, `data-contract-planted-errors.md`
Day 2 extend with STTM and SLAs	Yes, if Day 1 ran on time	`sttm.md`, `service-levels.md`
Day 2 Markdown to HTML	Yes, on a fast model with low effort first	`data-contract.html`
Day 2 contract to dbt and Airflow	Show the first minute live, then open the prepared project	`day2-platform/demos/contract-to-code/eod_prices/` and the `eod_prices_daily` DAG

Rule for every live generation: before accepting anything, read one section aloud and find the invented detail (a field, an owner, a revision history entry, an SLA number). There is always one. That is the lesson, not the generation.

Prompts

The prompt library is `courseware/common/prompts/README.md`: one entry per activity, the Copilot form first and the Claude form beneath when they differ. Learners may be given it as it is.

Diagnostic prompts to hand learners when they are stuck

These work in Copilot Chat inside the dev container and are safe to share.

- **Auth or connection error from dbt**: “Run the three-step Databricks auth check from `.github/copilot-instructions.md` and tell me which step fails before suggesting I log in again.” (The repo’s own instructions stop Copilot from sending people to re-authenticate when the token simply did not reach the shell.)
- **A failing dbt model**: “Here is the error and the compiled SQL in `target/compiled/…`. Explain the error in one paragraph, then propose the smallest change to the model, and show me the `dbt build --select <model>` command to check it.”
- **A failing test**: “Run `dbt test --select <model> --store-failures` is not available here; instead show me the SQL dbt compiled for this test and query the rows it returns, then tell me whether the data or the test is wrong.”

© 2026 Decoded Limited · decoded.com

- **Tracing a rule**: “Here is one STTM row and the model. Point to the line that implements the source column, the transformation and the null rule, and tell me which test guards it.”
- **Airflow DAG will not import**: “Run `astro dev run dags list-import-errors`, paste the first error, and tell me whether it is a Python error, a Cosmos rendering error, or a missing connection, and the one command to fix it.”
- **Cold start**: “`echo $DBT_CATALOG` is empty in my terminal. Walk me through `mise run setup:databricks`, a new terminal, and `astro dev restart --wait 5m`.”
- **Asking Catalyst about itself**: “What are the phases of the data product lifecycle, what does the build phase require, and which of those requirements have a skill?”

Rules of engagement (say them, then enforce them gently)

1. Review every generated line before running it; the room is here to learn what to check.
2. Never paste a token, a password or a client record into a chat. The running example is synthetic for that reason.
3. When the output is wrong, fix the input (the transcript, the contract, the context) before fixing the output by hand.
4. Prefer a specific skill to the concierge; the concierge is slow and does everything.
5. Start a new conversation per task; summarise to a file first.

© 2026 Decoded Limited · decoded.com